

מדריך מתודולוגיית פיתוח: מאיפה מתחילים ב-MVVM?

View-First מול ViewModel-First ו"שביל הזהב" המומלץ לתלמידים ב-.NET MAUI

קהל יעד: מגמת הנדסת תוכנה / תכנון

ארכיטקטורה: MVVM Best Practices

תחום: מתודולוגיית הנדסת תוכנה

פרק מתודולוגי עצמאי

1. הדילמה של המפתח המתחיל 🤔

נניח שיש לנו רעיון לאפליקציה (למשל, ניהול פלייליסט של סרטונים), וכבר הגדרנו את מודל הנתונים של הישות (`VideoItem`).
(. בדיוק בנקודה זו, כל תלמיד שואל את עצמו: מה הצעד הבא שלי בקוד?)

- האם לפתוח קודם כל את קובץ ה-XAML ולעצב את המסך (גישת **View-First**)?
- או שמא לכתוב תחילה את מחלקת ה-`ViewModel` שמנהלת את הנתונים והפעולות (גישת **ViewModel-First**)?

2. ניתוח השוואתי: שתי הגישות ⚖️

הגישה	היתרונות	המלכודות לתלמיד
גישת View-First (עיצוב המסך תחילה ב-XAML)	• משוב ויזואלי מיידי – רואים את התוצאה הגרפית על המסך מיד. • קל להגדיר מה צריך להיות במסך תוך כדי שרטוט פקדים.	• הפיתוי לשבור MVVM: דאבל-קליק על כפתור מוביל לכתיבת אירועי <code>Button_Clicked</code> בקוד האחורי. • Bindings "באוויר": כותבים <code>{Binding SaveCommand}</code> במסך לפני שהפקודה קיימת, ללא השלמה אוטומטית (IntelliSense), וקל מאוד לטעות באיות.
גישת ViewModel-First (הגדרת המצב והפקודות תחילה)	• בדיקתיות מלאה (Testability): הלוגיקה נבדקת ופועלת לפני שנוגעים ב-XAML. • ה-XAML הופך להלבשה בלבד: כל ה-Bindings נשענים על מאפיינים קיימים ועובדים עם IntelliSense מלא ו-Compiled Bindings.	• דורש יכולת הפשטה גבוהה – צריך לדמיין את התנהגות המסך מבלי לראות אותו עדיין בעיניים.

3. "שביל הזהב" המומלץ לתלמידים: תהליך 4 השלבים 🏆

כדי ליהנות משני העולמות – גם בהירות ויזואלית וגם משמעת ארכיטקטונית נכונה – מומלץ לעבוד לפי "שביל הזהב":

1 שרטוט מהיר על דף נייר (Wireframe) ב-2 דקות

לפני שפותחים Visual Studio: לוקחים דף ומציירים מלבן של סמארטפון. מסמנים איפה יושב סרגל הדפדוף, איפה שדות הקלט ואיזה כפתורים קיימים. השרטוט מעניק מענה מיידי לשאלה: מה המשתמש רואה ומה הוא עושה?

2 הגדרת המודל והשירות (Model & Service)

יוצרים את מחלקת הנתונים (`VideoItem`) ואת ממשק השירות (`IVideoService`) עם מתודות `Task` א-סינכרוניות. כעת יש לנו מקור מידע אמין.

3 מימוש ה-ViewModel (תרגום השרטוט ל-State ו-Commands)

עוברים על השרטוט הידני שציירנו בשלב 1 וגוזרים מתוכו:

- כל טקסט שמשתנה על המסך → הופך ל-`Property` ב-`ViewModel` עם `.OnPropertyChanged()`
- כל כפתור שהמשתמש יכול ללחוץ → הופך ל-`Command` ב-`ViewModel` עם תנאי `.CanExecute`

4 הלבשת המסך ב-XAML (View & Bindings)

פותחים את קובץ ה-XAML, מניחים את הפקדים, ומבצעים `{Binding}` למאפיינים ולפקודות שכבר קיימים ונבדקו ב-`ViewModel`.

4. דוגמה מוחשית מתוך הפרויקט: כפתור "נקה טופס" 💡

ניתוח מקרה בוחן (Case Study):

נניח שבשרטוט המסך שלך מופיע כפתור

"נקה טופס"

לפי

"שביל הזהב"

:

1. בשלב 3 (ב-ViewModel):

נגדיר פקודת `ClearFormCommand`. נכתוב עבורה מתודה שמאפסת את המאפיינים:

```
private void ExecuteClearForm()
{
    InputTitle = "";
    InputChannelName = "";
    InputUrl = "";
    InputDurationText = "";
}
```

2. בשלב 4 (ב-XAML):

נחבר את הכפתור ישירות:

```
<Button Text="נקה טופס" Command="{Binding ClearFormCommand}" />
```

ה-Binding מתחבר מיד ללוגיקה שכבר עובדת ומגובה ב-IntelliSense מלא!

5. שאלות מחשבה ובדיקה עצמית לתלמיד

שאלה 1: מניעת תקלות Binding

מדוע כתיבת ה-ViewModel לפני ה-XAML מונעת באגים של שגיאות כתיב (Typo) בשמות המאפיינים והפקודות?

שאלה 2: כתיבת בדיקות יחידה (Unit Tests)

אם נרצה לבדוק שפקודת הדפדוף `NextCommand` ננעלת כאשר מגיעים לסרטון האחרון, האם אנחנו חייבים להפעיל את האפליקציה ולראות את המסך בעיניים? הסבר לפי עקרונות ה-ViewModel-First.

שאלה 3: אתגר תכנוני

שרטט על דף מסך עם כפתור "שכפל סרטון". תאר אילו שינויים תבצע ב-ViewModel (אילו מאפיינים ופקודות תוסיף) לפני שתיגש לערוך את ה-XAML.